

Formal Languages and Compilers

(Linguaggi Formali e Compilatori)

prof. Luca Breveglieri

(prof. A. Morzenti)

Written exam - 17 September 2014 - Part I: Theory

WITH SOLUTIONS - FOR TEACHING PURPOSES HERE THE SOLUTIONS ARE WIDELY

COMMENTED - THE CANDIDATE SHOULD CORRECTLY ANSWER AND REASONABLY EXPLAIN WHY

NAME:

MATRICOLA:

SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

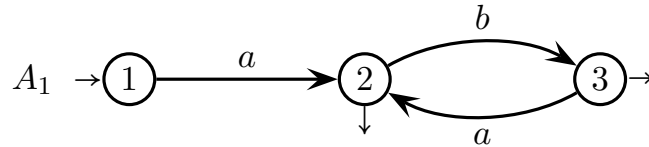
- The exam consists of two parts:
 - I (80%) Theory:
 1. regular expressions and finite automata
 2. free grammars and pushdown automata
 3. syntax analysis and parsing
 4. translation and semantic analysis
 - II (20%) Practice on Flex and Bison
- To pass the exam, the candidate must succeed in both parts (I and II), in one call or more calls separately, but within one year.
- To pass part I (theory) one must answer the mandatory (not optional) questions. Notice the full grade is achieved by answering the optional questions.
- The exam is open book (texts and personal notes are admitted).
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets nor replace the existing ones.
- Time: Part I (theory): 2h.15m - Part II (practice): 60m

1 Regular Expressions and Finite Automata 20%

1. Consider the following regular expression R_1 :

$$R_1 = (a \mid b)^* b$$

and the following deterministic finite automaton A_1 :



which are both defined over the alphabet $\{a, b\}$.

Answer the following questions:

- (a) By means of the Berry-Sethi method (*BS*), find a finite automaton A_2 such that $L(A_2) = L(R_1)$.
 - (b) Find a finite automaton A_3 such that $L(A_3) = L(A_1) \cap L(A_2)$.
 - (c) By means of the node elimination method (Brzozowski), find a regular expression R_2 such that $L(R_2) = L(A_3)$, i.e., the expression R_2 is equivalent to the automaton A_3 found before.
 - (d) (optional) By means of a systematic method of your choice, find a deterministic finite automaton A_4 such that $L(A_4) = \neg L(A_3)$, i.e., the language of A_4 is the complement of that of the automaton A_3 found before.
-

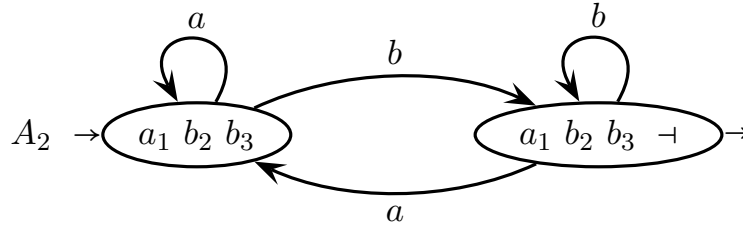
Solution

- (a) Here is the Berry-Sethi method applied to the regular expression R_1 . First the numeration, the initials and the followers:

$$R_{1\#} = (a_1 \mid b_2)^* b_3 \neg$$

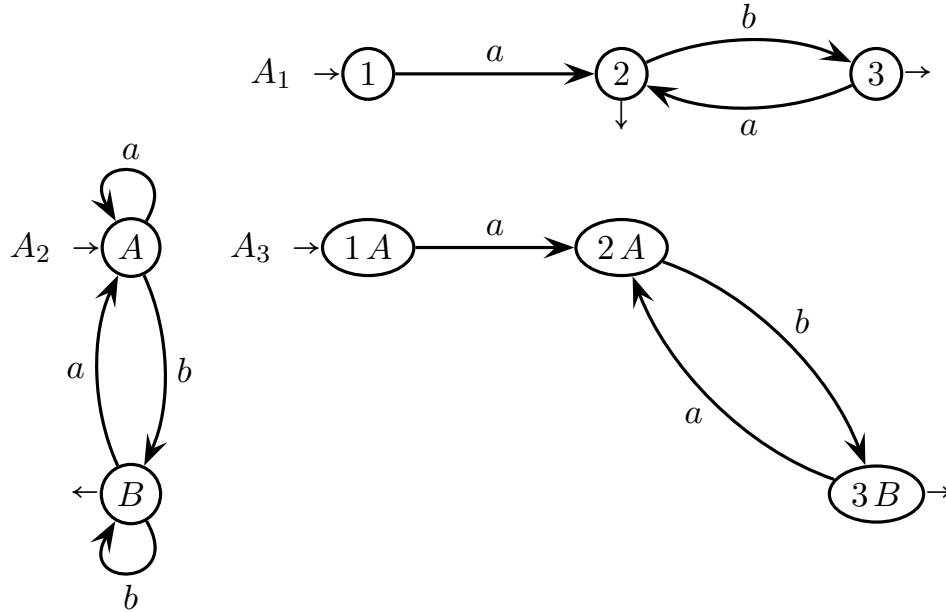
<i>initials</i>	$a_1 \ b_2 \ b_3$
<i>generator</i>	<i>followers</i>
a_1	$a_1 \ b_2 \ b_3$
b_2	$a_1 \ b_2 \ b_3$
b_3	$\neg$

Second, the deterministic automaton A_2 :



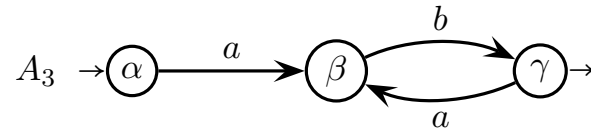
It is quite obvious that the automaton A_2 is minimal, as its two states are non-final and final, respectively, thus they are necessarily distinguishable.

- (b) Here is the intersection automaton A_3 (with the states of A_2 renamed):

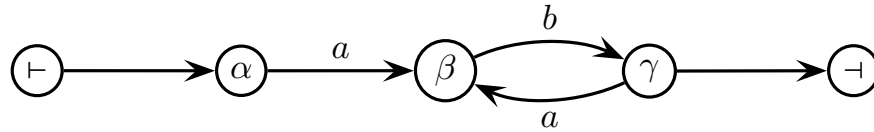


The automaton A_3 is in the reduced form (all its states are useful) and is deterministic. It is also minimal, as the two states that might be undistinguishable, namely states $1A$ and $2A$, have outgoing arcs with different labels.

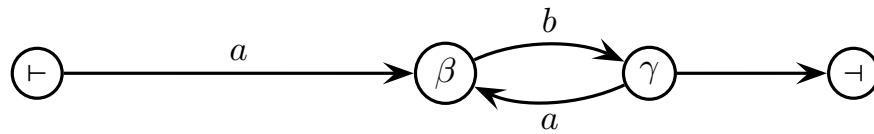
- (c) Here is the node elimination method (Brzozowsky) applied to automaton A_3 (with states renamed):



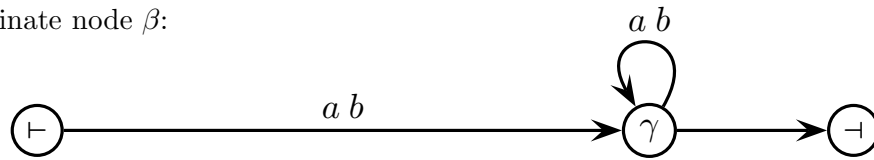
Put unique initial / final nodes with no ingoing / outgoing arcs:



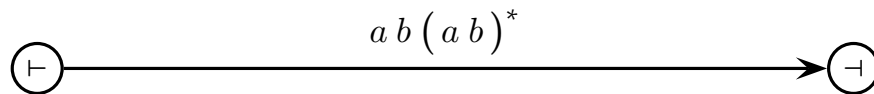
Eliminate node α :



Eliminate node β :



Eliminate node γ :

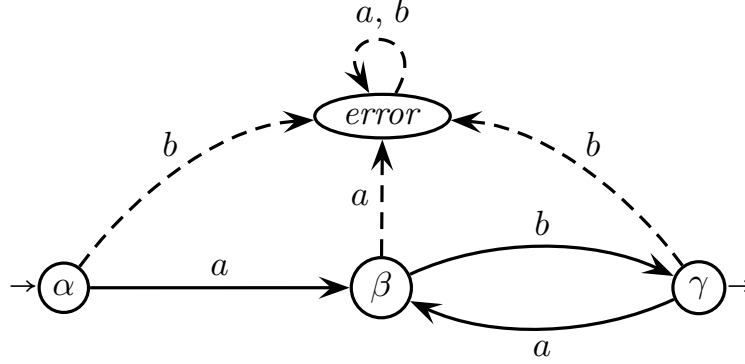


Therefore a regular expression R_2 is the following:

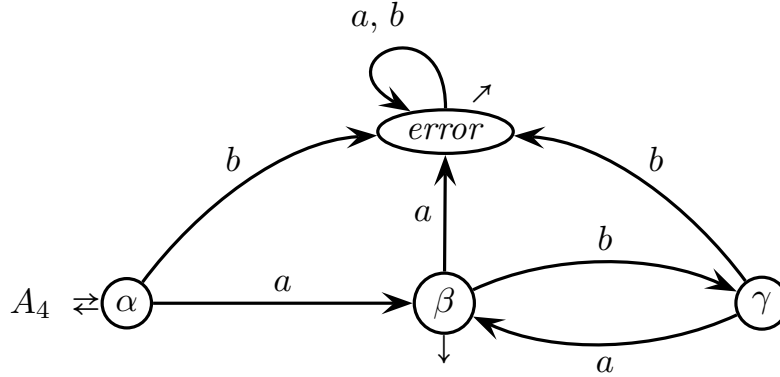
$$R_2 = a b (a b)^* = (a b)^+$$

It appears to be intuitively correct: have a close look at automaton A_3 . There may exist different formulations, although this one seems to be very simple.

- (d) Here is the construction of the complement automaton A_4 , applied to the (deterministic) automaton A_3 . First the error state:



Second, switching the non-final / final states:



Automaton A_4 is deterministic by construction, it is in the reduced form (all its states are useful), and it is minimal (the reader may check by himself).

Observation: it is easy to (partly intuitively) obtain a regular expression R_3 for automaton A_4 , like for instance this one (just list all the paths to a final node):

$$R_3 = \varepsilon \mid b \Sigma^* \mid a (b a)^* \mid a (b a)^* a \Sigma^* \mid a b (a b)^* b \Sigma^*$$

with the alphabet $\Sigma = \{a, b\}$, or (by factoring on the right):

$$R_3 = \varepsilon \mid a (b a)^* \mid \left((a b)^* b \mid a (b a)^* a \right) \Sigma^*$$

Notice that the expression R_3 looks more complex than expression R_2 , though it is not ambiguous. Of course there might be different and simpler formulations (for instance try again with a node elimination applied to automaton A_4).

2 Free Grammars and Pushdown Automata 20%

1. Consider the Dyck language L over the alphabet with round brackets $(,)$ and square brackets $[,]$.

Answer the following questions:

- (a) Write a *BNF* grammar G' , no matter if ambiguous, that generates the subset of language L the strings of which *do not contain* the digram $[($, i.e., they do not contain an open square bracket immediately followed by an open round bracket.
 - (b) (optional) Write a *BNF* grammar G'' , *not ambiguous*, such that in addition to the previous restriction it has the new restriction of not containing the digram $]($, i.e., the strings *do not contain* a closed square bracket immediately followed by an open round bracket.
-

Solution

- (a) To start, here is the standard (not ambiguous) Dyck grammar with round and square brackets:

$$S \rightarrow (S) S \mid [S] S \mid \varepsilon$$

Now, the basic idea may be that of splitting the axiom S into two nonterminals T and Q with (almost) disjoint languages, which generate strings that start by an open round bracket and an open square bracket (plus the empty string ε), respectively. Here it is, which still generates the complete Dyck language L :

$$\left\{ \begin{array}{l} S \rightarrow T \mid Q \mid \varepsilon \\ T \rightarrow (T) S \mid (Q) S \mid \varepsilon \\ Q \rightarrow [T] S \mid [Q] S \mid \varepsilon \end{array} \right.$$

The above grammar separates the derivations that generate round versus square nested brackets, whereas the concatenated ones are undifferentiated. Next, here is grammar G' , obtained from the above by deleting an alternative (axiom S):

$$G' \left\{ \begin{array}{l} S \rightarrow T \mid Q \mid \varepsilon \\ T \rightarrow (T) S \mid (Q) S \mid \varepsilon \\ Q \rightarrow [Q] S \mid \varepsilon \end{array} \right.$$

Since grammar G' misses the alternative rule $Q \rightarrow [T] S$, it cannot generate the strings that contain a digram ‘ $([$ ’, whereas it can still generate any other Dyck string type. Notice that grammar G' is ambiguous, as for instance there are both derivations $S \Rightarrow \varepsilon$ and $S \Rightarrow T \Rightarrow \varepsilon$ (and similarly for nonterminal Q).

- (b) We can refine grammar G' . First, disambiguate it in this way (axiom S):

$$\left\{ \begin{array}{l} S \rightarrow T \mid Q \mid \varepsilon \\ T \rightarrow (T) S \mid (Q) S \mid () S \\ Q \rightarrow [Q] S \mid [] S \end{array} \right.$$

By removing the null alternative rules $T \rightarrow \varepsilon$ and $Q \rightarrow \varepsilon$, the multiple null derivations cannot take place any more. Next, here is grammar G'' (axiom S):

$$G'' \left\{ \begin{array}{l} S \rightarrow T \mid Q \mid \varepsilon \\ T \rightarrow (T) S \mid (Q) S \mid () S \\ Q \rightarrow [Q] Q \mid [] Q \mid [Q] \mid [] \end{array} \right.$$

By refining the two alternative rules $Q \rightarrow [Q] S \mid [] S$ as the four alternative ones $Q \rightarrow [Q] Q \mid [] Q \mid [Q] \mid []$, i.e., by letting the right occurrence of axiom S go only into $Q \mid \varepsilon$ (and not into T), and so by excluding the derivations

$Q \Rightarrow [Q] T$ and $Q \Rightarrow [] T$ that would have a nonterminal T immediately following a closed square bracket $]$, it happens that the undesirable digram $]($ cannot be generated any more, whereas the strings that do not contain it can still be generated. As grammar G'' is a refinement of the one disambiguated before, it is not ambiguous either. This concludes the exercise.

Observation: it is possible to minimize the number of nonterminals, for both points (a) and (b). The three alternative rules $T \rightarrow (T) S \mid (Q) S \mid () S$ can be unified as the two ones $S \rightarrow (S) S \mid \varepsilon$, by factoring the axiom S on the right and then by observing that we already have rules $S \rightarrow T \mid Q \mid \varepsilon$. For point (a), the result is the grammar G'_1 below, equivalent to grammar G' and ambiguous as well (axiom S):

$$G'_1 \left\{ \begin{array}{l} S \rightarrow (S) S \mid Q \mid \varepsilon \\ Q \rightarrow [Q] S \mid \varepsilon \end{array} \right.$$

For point (b), the nonterminal Q can be made nullable again, thus compacting its alternative rules. The result is the grammar G''_1 below, equivalent to grammar G'' and not ambiguous either (axiom S):

$$G''_1 \left\{ \begin{array}{l} S \rightarrow (S) S \mid [Q] Q \mid \varepsilon \\ Q \rightarrow [Q] Q \mid \varepsilon \end{array} \right.$$

We could obtain this result more straightforwardly (though by using some more intuition) by noticing that the combined effect of forbidding both the digram $[($ and the digram $)]($ is to disallow all the Dyck strings that have round brackets inside of square ones. Clearly this is what the grammar G''_1 above does. Notice however that forbidding only the former digram $[($ still allows us to have certain strings with round brackets inside of square ones (this is what point (a) requires); for instance the string $[[] ()]$ is valid, but of course it gets invalid if the latter digram $)]($ is forbidden as well. It seems hard for a solution to exist with fewer nonterminals than two, although a formal proof of such a conjecture might be quite long to write.

2. The following specifications describe a candidate language for logical programming inspired to the well known Prolog language, although with a few simplifications:

- The program is a list (not empty) of *clauses*, and a clause can be simple or composite.
- A *simple* clause is a predicate terminated by a character ‘.’ (dot).
- A *composite* clause is a predicate, followed by a digram ‘:-’ (colon and dash), followed by a list (not empty) of predicates separated by a character ‘,’ (comma), and terminated by a character ‘.’ (dot).
- A *predicate* is an identifier made of *lowercase* letters, followed by a pair of matching round brackets ‘(’ and ‘)’’, which contain a list (not empty) of terms separated by a character ‘,’ (comma).
- A *term* is defined in three possible different ways, as follows:
 - either as an *atom*, i.e., an identifier made of *lowercase* letters
 - or as a *variable*, i.e., an identifier made of *uppercase* letters
 - or as a *functor*, i.e., an identifier made of *lowercase* letters, followed by a pair of matching round brackets ‘(’ and ‘)’’, which contain a list (not empty) of terms separated by a character ‘,’ (comma); clearly this last definition of term is *recursive*

Here is a sample program (the comments do not belong to the language):

```
man (socrates).          /* simple clause (has an atom)      */

int (zero).              /* simple clause (has an atom)      */

dead (X) :- man (X).     /* composite clause (has a variable) */

/* composite clause (has more variables)                      */
friend (X, Y) :- friend (X, Z), friend (Z, Y).

int (succ (X)) :- int (X). /* composite clause (has a functor) */

/* composite clause (has more functors)                      */
equal (add (X, succ (Y)), succ (add (X, Y))) :- int (X), int (Y).
```

Write an extended (*EBNF*) grammar, not ambiguous, that generates the logical programming language described before.

Solution

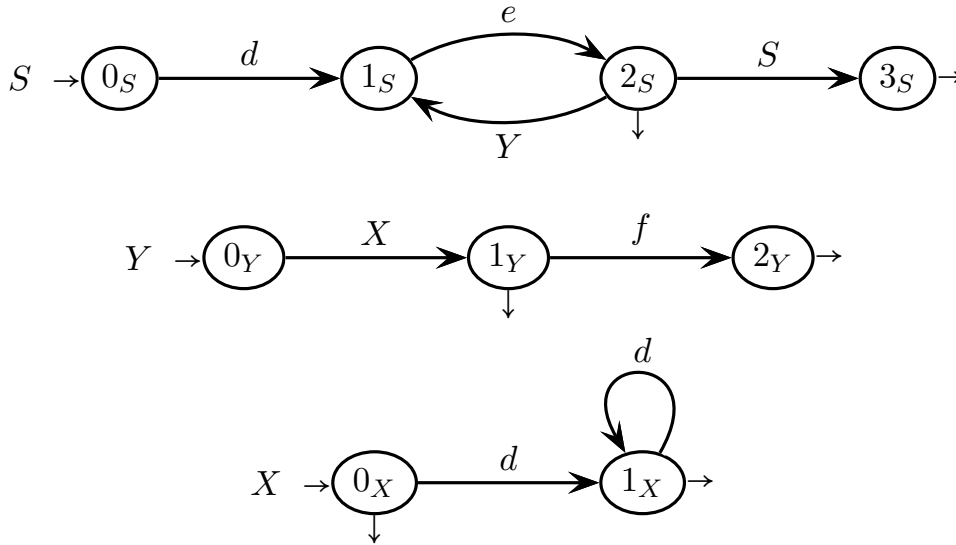
Here is a grammar that fulfills the specifications (axiom PROG):

$$\left\{ \begin{array}{l}
 \langle \text{PROG} \rangle \rightarrow (\langle \text{CLAUSE} \rangle ' .')^+ \\
 \langle \text{CLAUSE} \rangle \rightarrow \langle \text{S_CLAUSE} \rangle \mid \langle \text{C_CLAUSE} \rangle \\
 \hline
 \langle \text{S_CLAUSE} \rangle \rightarrow \langle \text{PRED} \rangle \\
 \langle \text{C_CLAUSE} \rangle \rightarrow \langle \text{PRED} \rangle ' :- ' (\langle \text{PRED} \rangle ' , ')^* \langle \text{PRED} \rangle \\
 \hline
 \langle \text{PRED} \rangle \rightarrow \langle \text{LCASE_ID} \rangle ' (' \langle \text{T_LST} \rangle ') ' \\
 \hline
 \langle \text{T_LST} \rangle \rightarrow (\langle \text{TERM} \rangle ' , ')^* \langle \text{TERM} \rangle \\
 \langle \text{TERM} \rangle \rightarrow \langle \text{ATOM} \rangle \mid \langle \text{VAR} \rangle \mid \langle \text{FUNCTOR} \rangle \\
 \hline
 \langle \text{ATOM} \rangle \rightarrow \langle \text{LCASE_ID} \rangle \\
 \langle \text{VAR} \rangle \rightarrow \langle \text{UCASE_ID} \rangle \\
 \langle \text{FUNCTOR} \rangle \rightarrow \langle \text{LCASE_ID} \rangle ' (' \langle \text{T_LST} \rangle ') ' \\
 \hline
 \langle \text{LCASE_ID} \rangle \rightarrow [a - z]^+ \\
 \langle \text{UCASE_ID} \rangle \rightarrow [A - Z]^+
 \end{array} \right.$$

This grammar is extended and not ambiguous. Here the square brackets denote an alphabetical interval. The grammar is designed by keeping in mind the order of the specifications. Each syntactic class models a language object. Class `T_LST` (term list) is introduced as it is used twice, whereas the predicate list of class `C_CLAUSE` occurs there only once and so it is unworthy deserving a specific class (one might choose differently). Notice that classes `PRED` and `FUNCTOR` are syntactically identical and could be unified. However, they differ semantically and so it may be appropriate to keep them separate also in the syntax, to help their semantic analysis later. Of course, there may be other and as good grammatical formulations.

3 Syntax Analysis and Parsing 20%

1. Consider the following recursive network $\mathcal{M}$ of machines, over the terminal alphabet $\{d, e, f\}$ and the nonterminal alphabet $\{S, X, Y\}$ (axiom S):



Answer the following questions:

- (a) Draw the complete pilot $\mathcal{P}$ of the network $\mathcal{M}$, say if the network is of type $ELR(1)$ and explain why.
- (b) Compute all the guide sets on the arcs (shift, call and final darts) of the network $\mathcal{M}$, say if the network is of type $ELL(1)$ and explain why.
- (c) Say if there exists another network $\mathcal{M}'$ equivalent to $\mathcal{M}$ (i.e., that defines the same language $L(\mathcal{M})$), but which has fewer machines; furthermore, if such a network $\mathcal{M}'$ exists, then say which is the minimum number of machines for such a network $\mathcal{M}'$ and explain why.
- (d) (optional) Consider the following valid string (i.e., such that it belongs to language $L(\mathcal{M})$):

$d e f e$

and simulate how to recognize such a string by means of the Earley method; please use the table prepared on the next page. Furthermore, draw the syntax tree of the string recognized.

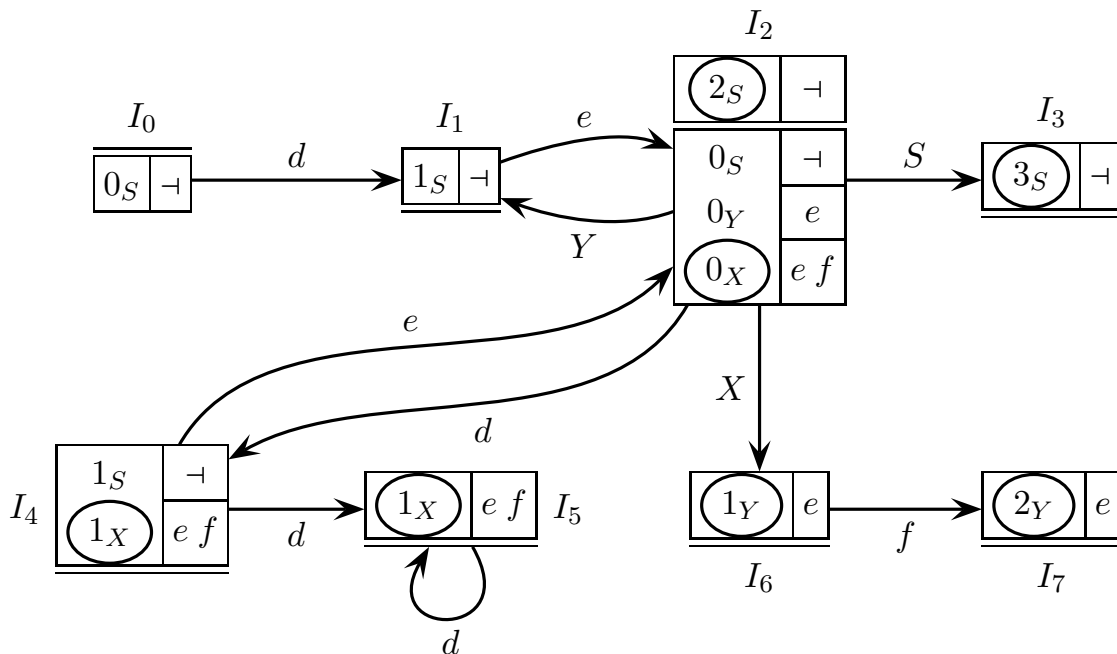
Earley vector of the valid string $d e f e$ (to be filled)
 (the number of rows is not significant)

0	d	1	e	2	f	3	e	4

syntax tree of the valid string $d e f e$ (to be drawn)

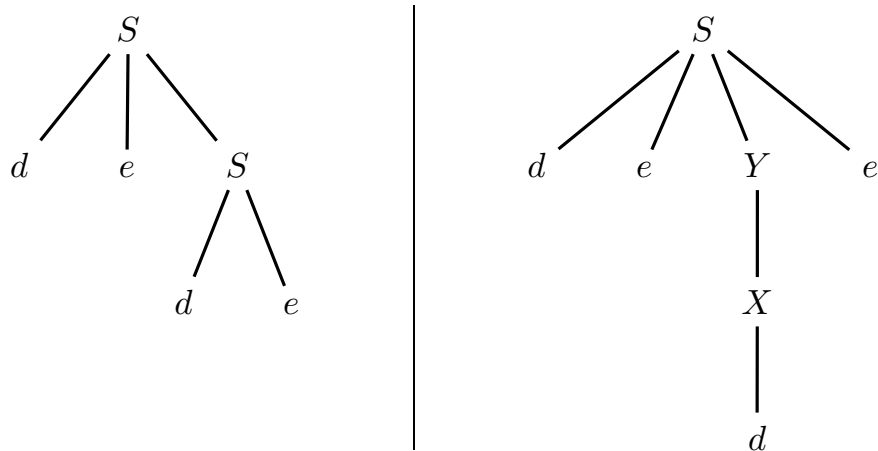
Solution

(a) Here is the complete pilot of network $\mathcal{M}$:



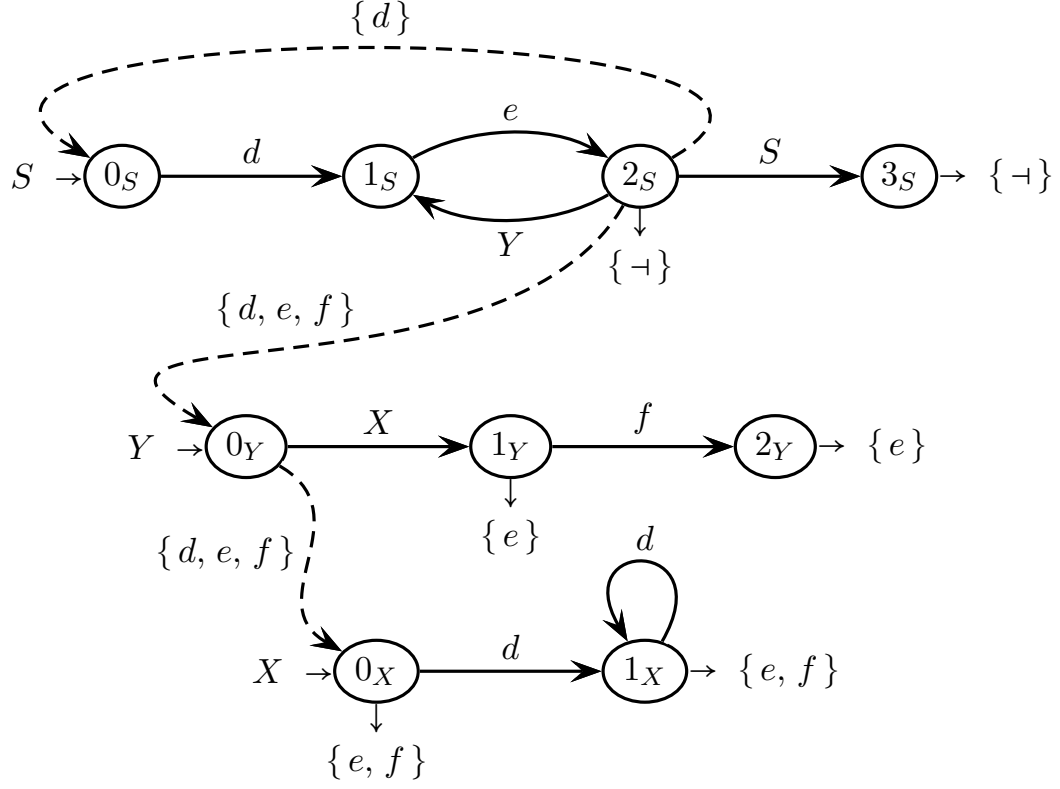
The pilot has 8 m-states. It has a shift-reduce conflict in the m-state I_4 (and no more conflicts in any other m-state), so the network $\mathcal{M}$ is not $ELR(1)$.

Observation (not requested): actually the network $\mathcal{M}$ is ambiguous. For instance, the string $dede$ has two syntax trees:



More generally, all the strings of type $(de)^+$ are ambiguous (and possibly other string types as well). Therefore clearly the network $\mathcal{M}$ is not $ELR(1)$.

- (b) Here is the network $\mathcal{M}$ with call arcs and guide sets (the guide sets on the terminal shift arcs coincide with the arc labels and here they are not shown):



The guide sets from the call arcs on state 2_S overlap (there are not any other overlaps anywhere). Therefore the network $\mathcal{M}$ is not $ELL(1)$.

Observation: of course, since the network M is not $ELR(1)$ (and is even ambiguous), it cannot be $ELL(1)$ either. The $ELL(1)$ analysis confirms this and shows more clearly where the recursive descent syntax analyzer fails to be deterministic.

- (c) The language $L(\mathcal{M})$ is regular. In fact:

$$L(\mathcal{M}) = L(S) = \left(d e (Y e)^* \right)^+$$

$$L(Y) = X (\varepsilon \mid f)$$

$$L(X) = \varepsilon \mid d d^* = d^*$$

therefore we obtain:

$$L(Y) = d^* (\varepsilon \mid f)$$

$$L(S) = \left(d e (d^* (\varepsilon \mid f) e)^* \right)^+ = L(\mathcal{M})$$

The finite automaton of $L(\mathcal{M})$ can be obtained by substitution of machines, or directly from the regular expression (e.g., use Berry-Sethi). Thus a network $\mathcal{M}'$ equivalent to $\mathcal{M}$ but consisting of only one machine is sufficient, and clearly this is the minimum number of machines, as one machine at least is necessary.

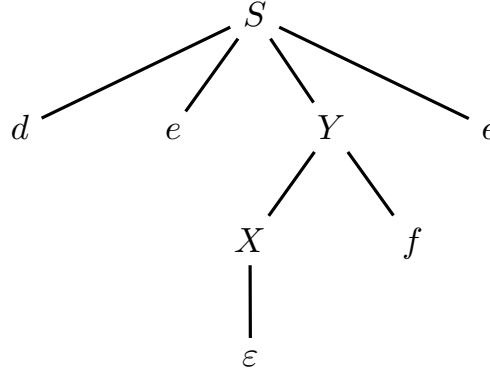
Observation: the network $\mathcal{M}$ is not $ELL(1)$, but the language is so, as it is regular; thus there exists an $LL(1)$ right-linear grammar that generates it.

(d) Here is the Earley analysis of the proposed (valid) sample string:

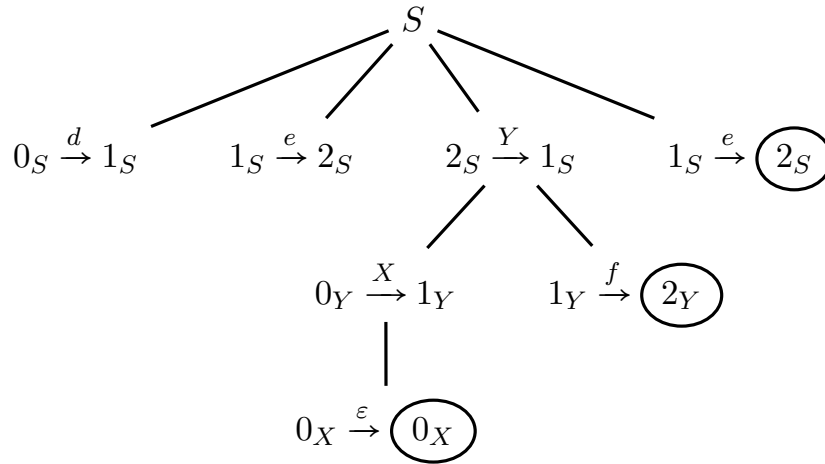
Earley vector of string $d e f e$ (filled)

0	d	1	e	2	f	3	e	4
0_S 0	1_S 0	(2_S) 0	(2_Y) 2	(2_S) 0				
		0_S 2	1_S 0	0_S 4				
		0_Y 2		0_Y 4				
		(0_X) 2		(0_X) 4				
		(1_Y) 2		(1_Y) 4				
		1_S 0		1_S 0				

Here is the corresponding syntax tree, which is unique as this particular string is not ambiguous (but remember that the network is ambiguous, as said before):



More in detail, here is the (same) syntax tree with the machine paths that are run by the parser (the final states of the paths are encircled):



The former syntax tree is simply the projection of the latter one onto the total alphabet (terminals and nonterminals) of the network.

4 Translation and Semantic Analysis 20%

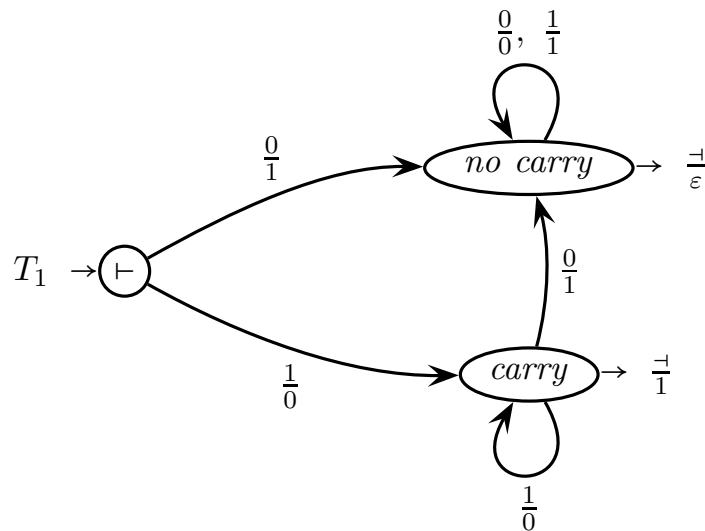
1. A binary string x represents a natural number according to the usual base 2 notation, but with the least significant bit on the left and the most significant bit on the right. For instance, the decimal value 13 (thirteen) is encoded by the binary string $x = 1011$. In this way, one can imagine to input the string x into a translator automaton starting from the least significant bit.

Answer the following questions:

- (a) Draw a sequential (finite) transducer T_1 (sometimes called *sequential machine*) that inputs a binary string x (according to the conventions explained before), with the restriction $|x| \geq 1$ (so the empty string ε is rejected), and that translates it into a binary string y (according to the same conventions) that encodes the numerical value of x incremented by *one*. For instance if the transducer inputs string $x = 1011$, then it outputs string $y = 0111$; and if it inputs $x = 111$, it outputs $y = 0001$ (see the different number of bits of x and y).
 - (b) Draw a sequential (finite) transducer T_2 as before (i.e., with the same conventions and the restriction $|x| \geq 2$), but that increments by *two*. For instance if the transducer inputs string $x = 1011$, then it outputs string $y = 1111$; and if it inputs $x = 111$, it outputs $y = 1001$ (see the different number of bits of x and y).
 - (c) (optional) Write a syntactic translation scheme G_s , G_d (or a translation grammar G_τ) that realizes the translation described at the previous point (b) and is deterministic, and explain why it is correct and deterministic.
-

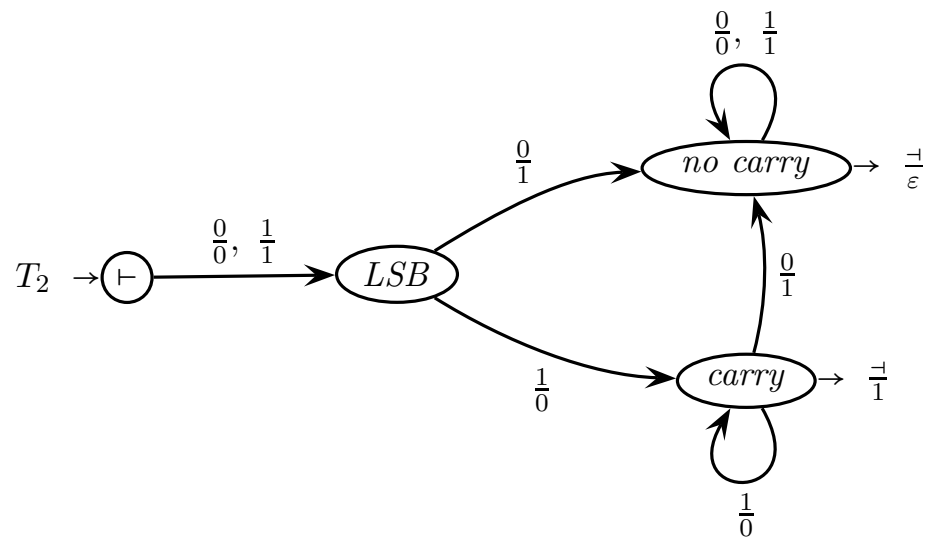
Solution

(a) Here is the sequential translator T_1 :



The names of the states are self-explanatory (just think of how binary arithmetic computes addition). Notice that this translator is deterministic, as the underlying recognizer is so.

(b) Here is the sequential translator T_2 :



The state names are self-explanatory (*LSB* stands for *Least Significant Bit*). The translator T_2 is built on translator T_1 , with one more state *LSB* to distinguish the first input bit from the more significant ones that are input next. Notice that this translator is also deterministic, as the underlying recognizer is so.

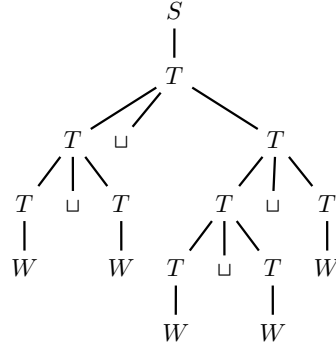
- (c) To obtain a syntactic translation scheme (or grammar) for point (b), it suffices to transform the finite-state translator T_2 into a right-linear (translation) grammar, which will be of type $LL(1)$ as such a translator is deterministic. Here is the *BNF* translation grammar G_τ (axiom $\langle \neg \rangle$):

$$G_\tau \left\{ \begin{array}{l} \langle \neg \rangle \rightarrow 0 \{0\} \langle \text{LSB} \rangle \\ \langle \neg \rangle \rightarrow 1 \{1\} \langle \text{LSB} \rangle \\ \hline \langle \text{LSB} \rangle \rightarrow 0 \{1\} \langle \text{NO_CARRY} \rangle \\ \langle \text{LSB} \rangle \rightarrow 1 \{0\} \langle \text{CARRY} \rangle \\ \hline \langle \text{NO_CARRY} \rangle \rightarrow 0 \{0\} \langle \text{NO_CARRY} \rangle \\ \langle \text{NO_CARRY} \rangle \rightarrow 1 \{1\} \langle \text{NO_CARRY} \rangle \\ \langle \text{NO_CARRY} \rangle \rightarrow \varepsilon \{\varepsilon\} \\ \hline \langle \text{CARRY} \rangle \rightarrow 0 \{1\} \langle \text{NO_CARRY} \rangle \\ \langle \text{CARRY} \rangle \rightarrow 1 \{0\} \langle \text{CARRY} \rangle \\ \langle \text{CARRY} \rangle \rightarrow \varepsilon \{1\} \end{array} \right.$$

The nonterminal names are self-explanatory and in one-to-one correspondence with the translator states. The graph brackets isolate the terminals of the target alphabet (which is the same as the source one). This translation grammar G_τ is $LL(1)$, as the underlying source grammar is so.

2. A syntactic support G generates a text (not empty) made of words W (it is not necessary to expand this nonterminal), which are separated by a blank character '□'. Here is the support G of the text (axiom S), with a sample syntax tree aside:

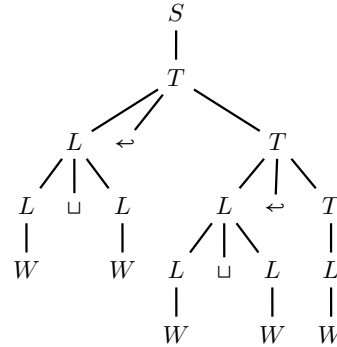
$$G \left\{ \begin{array}{lcl} S & \rightarrow & T \\ T & \rightarrow & T \sqcup T \\ T & \rightarrow & W \end{array} \right.$$



Answer the following questions (use the tables and trees on the next pages):

- (a) By means of an integer attribute $w \geq 1$, write an attribute grammar G_1 that computes how many words there are in the text. The result is expressed by w in the root S . Decorate the sample syntax tree with the values of w .
- (b) By means of the attribute w as before and of a new integer attribute $p \geq 0$, write an attribute grammar G_2 that computes how many words precede each word. The results are expressed by p in the respective leaves W . Decorate the sample syntax tree with the values of w e p .
- Furthermore say if the attribute grammar G_2 is of type one-sweep or not, and shortly explain why.
- (c) (optional) The syntactic support G is replaced by a new support G' that can split the text into lines (not empty), as follows (axiom S), where the terminal ' $\leftarrow$ ' represents a newline (there is a sample tree aside):

$$G' \left\{ \begin{array}{lcl} S & \rightarrow & T \\ T & \rightarrow & L \leftarrow T \\ T & \rightarrow & L \\ L & \rightarrow & L \sqcup L \\ L & \rightarrow & W \end{array} \right.$$



By means of the attribute w as before and of a new integer attribute $r \geq 0$ (and even more if it helps), write an attribute grammar G_3 (on the support G') that computes how many words there are in the text and how many lines precede each word. The results are expressed by w in the root S and by r in the respective leaves W . Decorate the sample syntax tree with the values of w and r .

Furthermore say if the attribute grammar G_3 is of type one-sweep or not, and shortly explain why.

attributes already assigned to be used for support G

fill the field *type*

<i>type</i>	<i>name</i>	<i>domain</i>	<i>nonterm.</i>	<i>meaning</i>
	w	integer ≥ 1	S, T	number of words
	p	integer ≥ 0	T, W	number of words that precede the current word

attributes already assigned to be used for support G'

and possible auxiliary attributes to be added (if it helps)

fill the field *type*

<i>type</i>	<i>name</i>	<i>domain</i>	<i>nonterm.</i>	<i>meaning</i>
	<i>w</i>	integer ≥ 1	<i>S, T, L</i>	number of words
	<i>r</i>	integer ≥ 0	<i>T, L, W</i>	number of lines that precede (the line where it is located) the current word

$$1: S_0 \rightarrow T_1$$

$$2: T_0 \rightarrow T_1 \sqcup T_2$$

$$3: T_0 \rightarrow W_1$$

$$1: S_0 \rightarrow T_1$$

$$2: T_0 \rightarrow T_1 \sqcup T_2$$

$$3: T_0 \rightarrow W_1$$

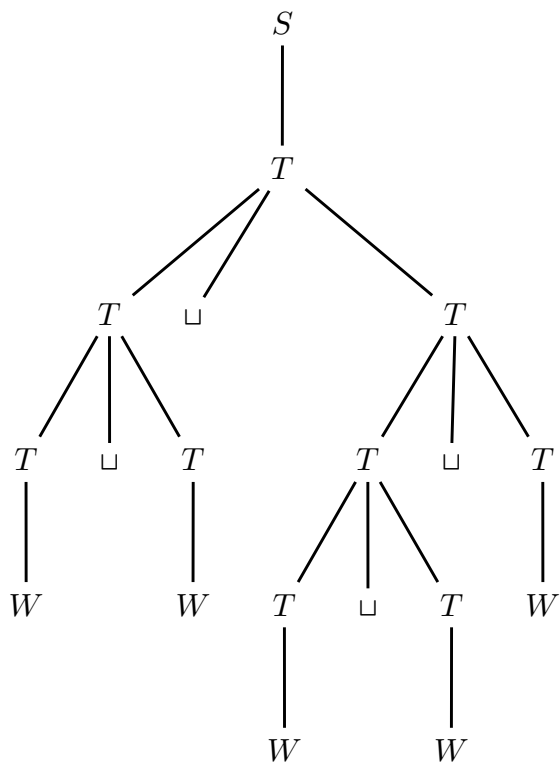
$$1: S_0 \rightarrow T_1$$

$$2: T_0 \rightarrow L_1 \leftarrow T_2$$

$$3: T_0 \rightarrow L_1$$

$$4: L_0 \rightarrow L_1 \sqcup L_2$$

$$5: L_0 \rightarrow W_1$$

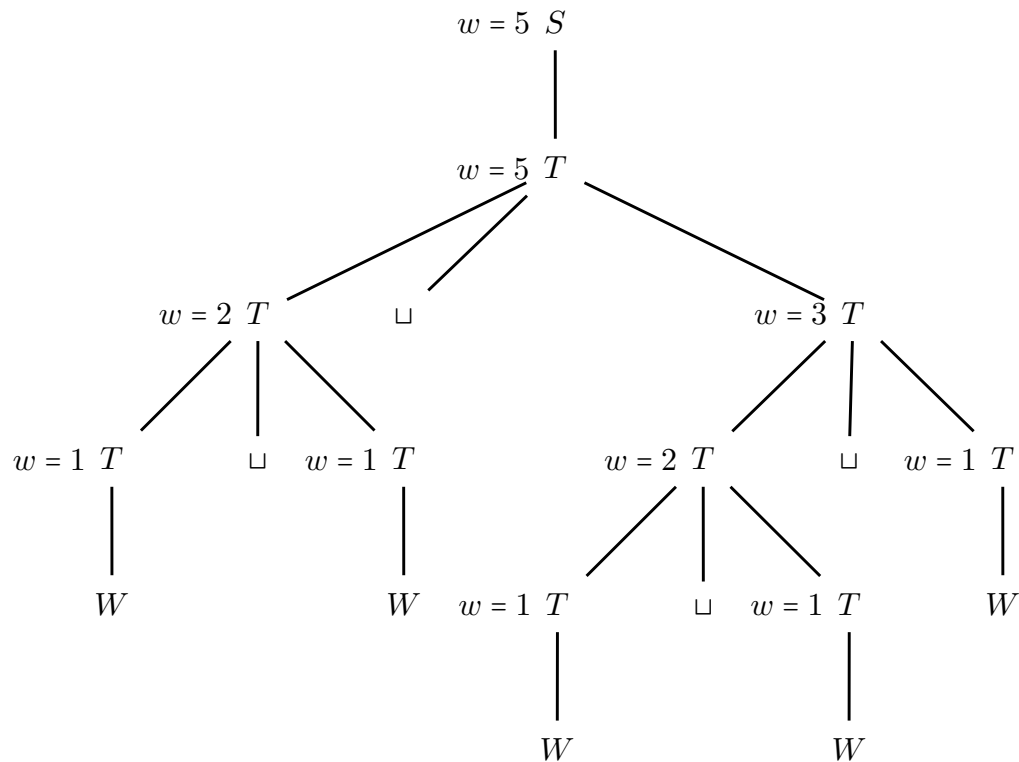


Solution

(a) Here is grammar G_1 (on the support G) with the computation of attribute w :

<i>type</i>	<i>name</i>	<i>domain</i>	<i>nonterm.</i>	<i>meaning</i>
left	w	integer ≥ 1	S, T	number of words
#	<i>syntax</i>		<i>semantics</i> - question a	
1:	$S_0 \rightarrow T_1$		$w_0 = w_1$	(result)
2:	$T_0 \rightarrow T_1 \sqcup T_2$		$w_0 = w_1 + w_2$	
3:	$T_0 \rightarrow W_1$		$w_0 = 1$	

Here is the decoration of the first syntax tree with attribute w (left):



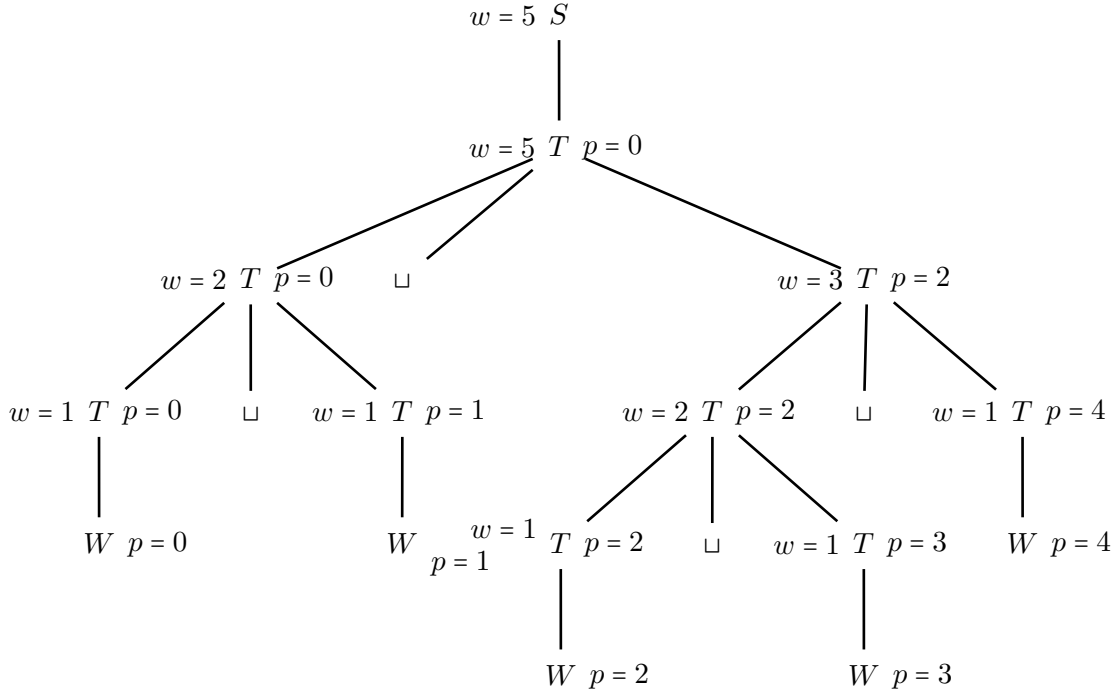
The final result, i.e., the total number of words in the text, is shown by the attribute w associated with the tree root S .

- (b) Here is grammar G_2 (on the support G) with the computation of attribute p (along with that of attribute w , which is repeated as before):

<i>type</i>	<i>name</i>	<i>domain</i>	<i>nonterm.</i>	<i>meaning</i>
right	p	integer ≥ 0	T, W	number of words that precede the current word

#	<i>syntax</i>	<i>semantics</i> - question b
1:	$S_0 \rightarrow T_1$	$p_1 = 0$ $w_0 = w_1$
2:	$T_0 \rightarrow T_1 \sqcup T_2$	$p_1 = p_0$ $p_2 = p_0 + w_1$ $w_0 = w_1 + w_2$
3:	$T_0 \rightarrow W_1$	$p_1 = p_0$ (result) $w_0 = 1$

Here is the decoration of the second syntax tree with attribute w (left), which coincides with what is shown before on the first tree, and attribute p (right):



The final results, i.e., the total number of words in the text and the numbers of words that precede each word of the text, are shown by the attribute w associated with the tree root S and by the attributes p associated with each tree leaf W .

Yes, grammar G_2 is one-sweep: the crucial point is the semantic rule $p_2 = p_0 + w_1$, but attribute p_2 can be computed after exploring the child subtree T_1 on the left

and thus computing attribute w_1 ; in all the other semantic rules, the attributes p and w are independent, and therefore they are computable in any order, so for instance first (top-down) the right attribute p and then (bottom-up) the left attribute w , as it is required by the one-sweep grammar model. More formally, one could check the one-sweep condition and see it is satisfied.

- (c) Here is grammar G_3 (on the support G') with the computation of attribute r (along with that of attribute w , which is essentially the same as before):

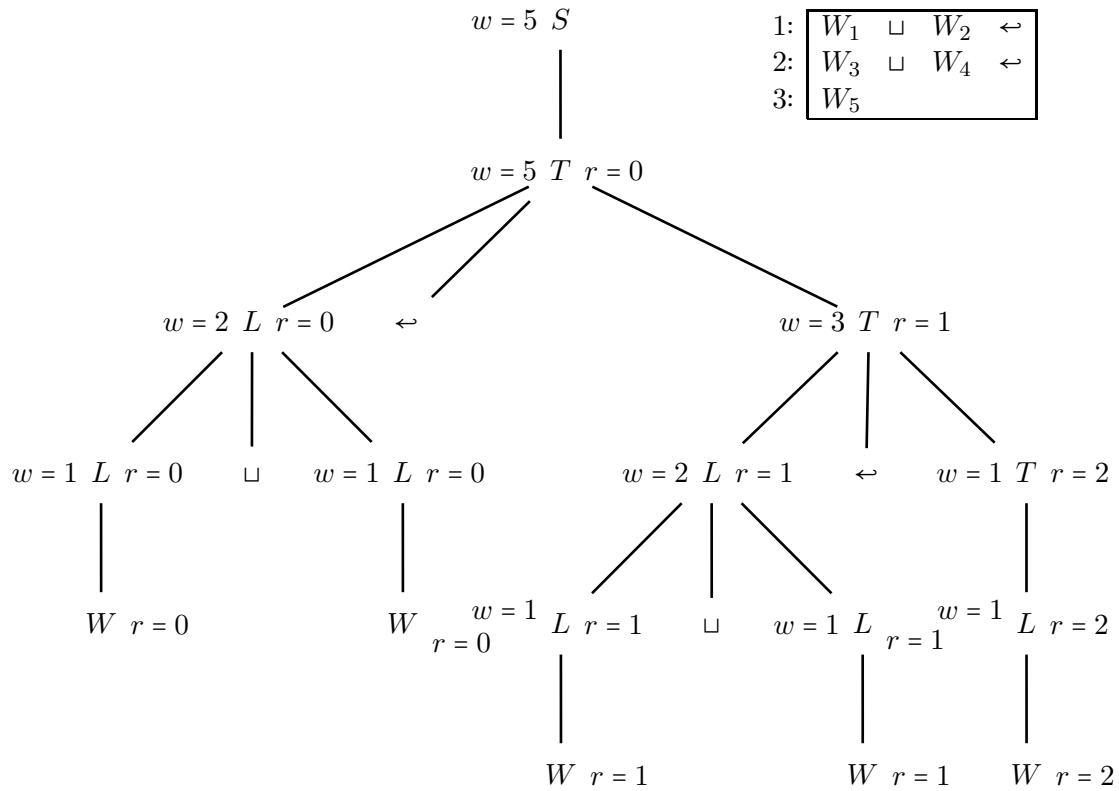
<i>type</i>	<i>name</i>	<i>domain</i>	<i>nonterm.</i>	<i>meaning</i>
	w	integer ≥ 1	S, T, L	number of words
	r	integer ≥ 0	T, L, W	number of lines that precede the (line where it is located the) current word

#	<i>syntax</i>	<i>semantics</i> - question c
1:	$S_0 \rightarrow T_1$	$r_1 = 0$ $w_0 = w_1$ (result)
2:	$T_0 \rightarrow L_1 \leftarrow T_2$	$r_1 = r_0$ $r_2 = r_0 + 1$ $w_0 = w_1 + w_2$
3:	$T_0 \rightarrow L_1$	$r_1 = r_0$ $w_0 = w_1$
4:	$L_0 \rightarrow L_1 \sqcup L_2$	$r_1 = r_0$ $r_2 = r_0$ $w_0 = w_1 + w_2$
5:	$L_0 \rightarrow W_1$	$r_1 = r_0$ (result) $w_0 = 1$

It is not necessary to introduce any more auxiliary attributes.

Yes, grammar G_3 is one-sweep: the two attributes w (left) and r (right) are fully independent, so it is possible to compute first attribute r and then attribute w , as it is required by the one-sweep grammar model.

Here is the decoration of the third syntax tree, with attribute w (left) and attribute r (right) (aside it is shown the separation and numbering of the text into lines and words):



The final results, i.e., the total number of words in the text and the numbers of lines that precede each word of the text, are shown by the attribute w associated with the tree root S and by the attributes r associated with each tree leaf W .